

Le Fin

Project: P05 – THIS IS JEOPARDY!!!!!!

Team: Pomegranates

Roster: Jason Chan (PM), Artemis Lee, Ethan Cheung

PROJECT DESCRIPTION:

Users will be able to play a game of Jeopardy with friends. They may use pre-generated boards created from our database of questions, or they may make their own customizable gameboard. Upon playing a game, the user will reveal clues and take keyboard inputs from the players. A player will gain/lose points depending on if they give the correct answer. Obviously, whoever has the most points wins. Have fun!

PROGRAM COMPONENTS + EXPLANATION:

Python files

- `__init__.py`: The main file; serves app
- `data_setup.py`: Handles parsing of CSV file and creation of Sqlite3 database tables
- `data.py`: Handles the data in the Sqlite3 grocery database and the user data (all stored in `data.db`)

Templates

- `login.html`: The user can login to their account or register if they don't have one. Redirects to the homepage.
- `register.html`: The user will be able to register. Redirects to the homepage.
- `home.html`: The homepage, which will allow users to start a new game with created boards/pre-generated boards, or start creating/editing a custom board, or view the leaderboards.
- `leaderboards.html`: Displays the leaderboard for most overall points, wins, runner-ups, and losses.
- `create_board.html`: When creating a new custom board, the user will have to give it a unique title before adding in

their clues. They also can edit the boards that they already have.

- *create.html*: Where users edit a custom board. They may click on any clue/category name to edit. They may also save or publish the board.
- *edit.html*: Where users edit an individual slot of the board. If it's a category, they can change the category name. If it's a clue, they can change the question, answer, up to three incorrect answers, and point values. Redirects to the create page.
- *find_or_create_room.html*: When starting a new game, users must enter a room code. This joins the other players with that same room code, or creates a new room if it doesn't exist. Redirects to the lobby page.
- *lobby.html*: Displays the list of players in the active room. Users can leave the room, which redirects to *find_or_create_room*. When there's three players, they can start the game, which redirects everyone in that room to *new_game*.
- *new_game.html*: When starting a new game, player 1 can choose to play a published board, or to play a generated board from the Trivia API. Redirects everyone in that room to *game.html*.
- *game.html*: The player in control of the board selects a clue. By default it is player 1. Redirects everyone in that room to *buzzer.html*.
- *buzzer.html*: Slowly displays the clue. After, users in a room can buzz in to answer. After everyone clicks the buzzer, the fastest player gets to answer the question multiple choice style. If they're wrong, then the second fastest player answers. Then the third. After, redirects everyone in that room back to *game.html*, with the clue no longer being able to be selected.
- *end_game.html*: When the game ends, displays the results and disconnects the lobby.

JavaScript files

- *buzzer.js*: Handles the buzzer inputs during a game.

CSS files/FEF

- We will be using Tailwind because it is awesome. We will also be using a few CSS files for some specific things.

DATABASE ORGANIZATION:

We will be using SQLite3 to organize our data; more specifically the user information, their saved boards, and current game stats. We will not be using any databases from external sources.

Tables:

users

Variable Type	Variable Name	Variable Attribute(s)
TEXT	username	PK NOT NULL
TEXT	password	NOT NULL
TEXT	bio	NOT NULL
INT	total_points	NOT NULL
INT	wins	NOT NULL
INT	runnerups	NOT NULL
INT	losses	NOT NULL

board

Variable Type	Variable Name	Variable Attribute(s)
TEXT	title	NOT NULL
TEXT	username	NOT NULL
INT	published	NOT NULL

INT	row	NOT NULL
INT	column	NOT NULL
STRING	question_category	NOT NULL
STRING	answer	
STRING	wrong1	
STRING	wrong2	
STRING	wrong3	
INT	point_value	
INT	chosen	NOT NULL

game

Variable Type	Variable Name	Variable Attribute(s)
TEXT	game_id	PK NOT NULL
TEXT	board	NOT NULL
TEXT	player1	NOT NULL
TEXT	player2	NOT NULL
TEXT	player3	NOT NULL
TEXT	player_in_control	NOT NULL
INT	points1	NOT NULL
INT	points2	NOT NULL
INT	points3	NOT NULL

lobbies

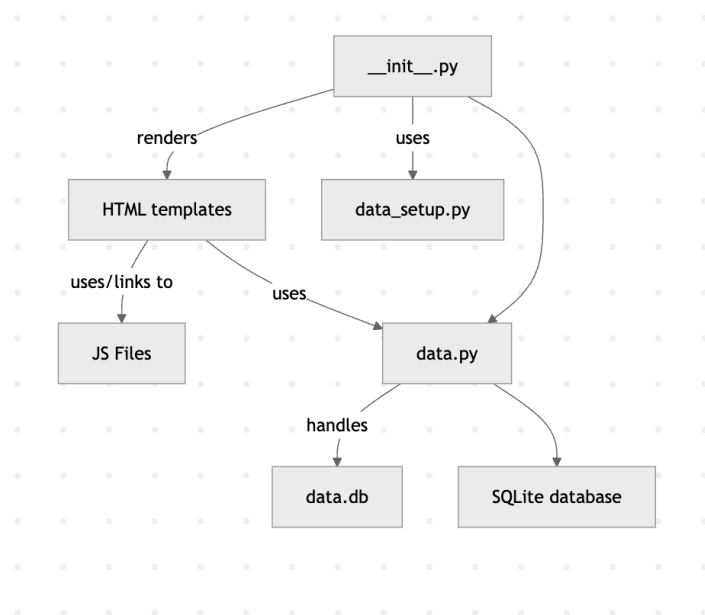
Variable Type	Variable Name	Variable Attribute(s)
---------------	---------------	-----------------------

TEXT	lobby_id	PK NOT NULL
TEXT	player1	NOT NULL
TEXT	player2	NOT NULL
TEXT	player3	NOT NULL

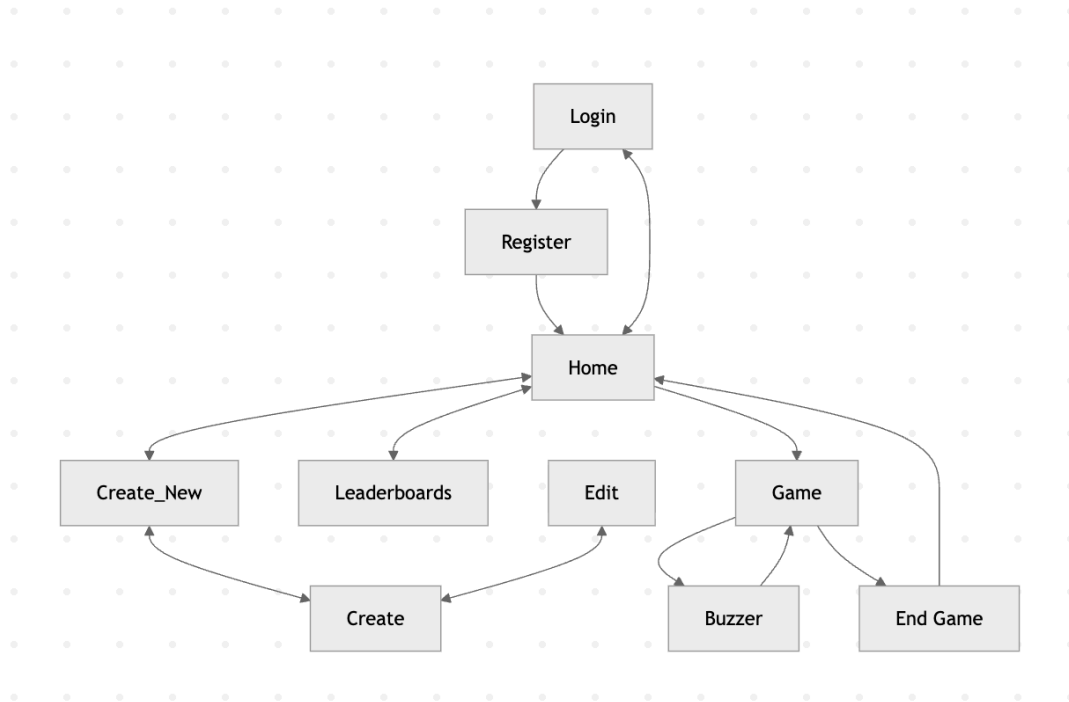
APIs:

Open Trivia API: Provides a whole bunch of trivia questions as requested. Responsive and easy to filter, but only outputs 50 questions at a time. No sign up required, just provides a link.

COMPONENT MAP:



FRONT END DIAGRAM:



TIMELINE:

WEEK 1:

- Get basic flask app and HTML template structure working
- Login/Register
- data_setup.py completed or close to complete

WEEK 2:

- Functions for data.py completed
- OpenTDB API implementation
- Users should be able to create/edit game boards

WEEK 3:

- Playing the game should be functional
 - Displaying clues
 - Buzzer inputs
 - Keeping track of stats throughout the game
- Any stretch goals we may have time for
 - Being able to play other users' custom boards

- Leaderboards for various stats (wins/losses, total points, correct responses, etc.

TASK DELEGATION (REALLY SUBJECT TO CHANGE! DEVOS ARE NOT FULLY RESTRICTED TO ASSIGNED TASKS!):

Task	Devo (s)	Deadline
Html stuff (page rendering)	DEC	05-18 @ 11:06
Tailwind stuff (styling)	DEC	06-14 @ 23:59
Database handling (making and applying data_setup.py and data.py)	DJC	05-21 @ 11:06
API stuff (implementing OpenTDB API)	DJC	05-21 @ 11:06
Python stuff (making flask work)	DAL	05-15 @ 23:59
Javascript stuff (making buzzer work)	DAL	05-26 @ 11:06
Socket stuff (joining/leaving lobbies, handling data throughout the game)	EVERYONE	06-14 @ 23:59
Devlog	EVERYONE	06-01 @ 08:00